torch.fmod#

https://pytorch.org/docs/stable/generated/torch.fmod.html

Description:

Applies C++'s std::fmod entrywise. The result has the same sign as the dividend input and its absolute value is less than that of other. This function may be defined in terms of torch.div() as Supports broadcasting to a common shape, type promotion, and integer and float inputs. When the divisor is zero, returns NaN for floating point dtypes on both CPU and GPU; raises RuntimeError for integer division by zero on CPU; Integer division by zero on GPU may return any value.

Function Signature

```
torch.fmod(input, other, *, out=None) → Tensor#
Parameters
Keyword Arguments
```

Parameters

Keyword

out (Tensor, optional) – the output tensor.

Code Examples

```
torch.fmod(a, b) == a - a.div(b, rounding_mode="trunc") * b
```

```
>>> torch.fmod(torch.tensor([-3., -2, -1, 1, 2, 3]), 2)
tensor([-1., -0., -1.,  1.,  0.,  1.])
>>> torch.fmod(torch.tensor([1, 2, 3, 4, 5]), -1.5)
tensor([1.0000, 0.5000, 0.0000, 1.0000, 0.5000])
```

Notes & Warnings

NOTE Note When the divisor is zero, returns NaN for floating point dtypes on both CPU and GPU; raises RuntimeError for integer division by zero on CPU; Integer division by zero on GPU may return any value.

NOTE Note Complex inputs are not supported. In some cases, it is not mathematically possible to satisfy the definition of a modulo operation with complex numbers.

NOTE See also torch.remainder() which implements Python's modulus operator. This one is defined using division rounding down the result.

Detailed Documentation

Documentation

Applies C++’s std::fmod entrywise. The result has the same sign as the dividend input and its absolute value is less than that of other.

This function may be defined in terms of torch.div() as

Supports broadcasting to a common shape, type promotion, and integer and float inputs.

When the divisor is zero, returns NaN for floating point dtypes on both CPU and GPU; raises RuntimeError for integer division by zero on CPU; Integer division by zero on GPU may return any value.

Complex inputs are not supported. In some cases, it is not mathematically possible to satisfy the definition of a modulo operation with complex numbers.

torch.remainder() which implements Python’s modulus operator. This one is defined using division rounding down the result.

input (Tensor) – the dividend

other (Tensor or Scalar) – the divisor

out (Tensor, optional) – the output tensor.